Constraints in SQL

Presented by: Isha Landge

What are Constraints in SQL?

SQL Constraints are rules applied to table columns to enforce data integrity and accuracy.

They ensure that:

- Invalid data cannot be inserted
- Data remains consistent
- Relationships between tables are maintained

👉 **Constraints help maintain Data Integrity**

Why Do We Need Constraints?

WITHOUT Constraints

- ✗ Duplicate data can enter
- ✗ NULL values may cause issues
- ✗ Wrong data types may be inserted
- ✗ Relationships may break

→

WITH Constraints

- ✓ Data is reliable
- ✓ Errors are minimized
- ✓ Database becomes secure

Types of Constraints — Step 1: The Problem

Let's first create a table WITHOUT any constraints — and see what can go wrong.

```sql
-- Table with NO constraints
CREATE TABLE Employees (
  EmpID   INT,
  Name    VARCHAR(100),
  Age     INT,
  Email   VARCHAR(150),
  DeptID  INT
);
```

Sample bad data MySQL would happily accept:

```sql
INSERT INTO Employees VALUES (101, NULL, -5,
'isha@email.com', 999);
```

⚠ Problems This Causes

- ✗ Name left as NULL (empty)
- ✗ Age = -5 or 999 → accepted!
- ✗ Duplicate emails allowed
- ✗ DeptID = 999 (doesn't exist)
- ✗ No default for missing fields

Types of SQL Constraints

NOT NULL

Column must have a value

UNIQUE

All values must be different

PRIMARY KEY

NOT NULL + UNIQUE identifier

FOREIGN KEY

Links two tables together

CHECK

Values must meet a condition

DEFAULT

Sets a default value

NOT NULL Constraint

Ensures a column cannot have a NULL value — every row MUST have a value in that column.

```
Syntax:
CREATE TABLE table_name(
     column_name datatype NOT NULL
);
```

```
Example:
CREATE TABLE Employees(
     EmpID   INT   NOT NULL,
     Name  VARCHAR(100) NOT NULL
);
```

💡 **EmpID and Name must always have values — empty entries are rejected.**

UNIQUE Constraint

Ensures all values in a column are different — no two rows can share the same value.

```
Syntax:
CREATE TABLE table_name(
     column_name datatype UNIQUE
);
```

```
Example:
CREATE TABLE Users (
    UserID  INT,
    Email   VARCHAR(100) UNIQUE
);
```

💡 **No two users can have the same email address.**

PRIMARY KEY Constraint

PRIMARY KEY combines:

NOT NULL **UNIQUE**

→ Uniquely identifies each record in a table

```
KEY RULES FOR PRIMARY KEY:

❌ Cannot be NULL
❌ Cannot have duplicate values
✓Only one PRIMARY KEY per table
✓ Can be single-column or composite (multi-column)
```

```
Syntax:
CREATE TABLE table_name (
column_name datatype PRIMARY KEY
)
```

```
Example:
CREATE TABLE Students (
    RollNo  INT PIMARY KEY,
    Name   VARCHAR(100)
);
```

✗ **RollNo cannot be NULL**

✗ **RollNo cannot be duplicate**

PRIMARY KEY = NOT NULL + UNIQUE — the most important constraint!

Composite key Constraint

A **Composite Key** is a PRIMARY KEY made using **more than one column**.
It is used when **a single column is not enough** to uniquely identify a record.

```
Syntax:
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    PRIMARY KEY (column1, column2)
);
```

```
Example:
CREATE TABLE OrderDetails (
   OrderID   INT,
   ProductID INT,
   Quantity  INT,
   PRIMARY KEY (OrderID, ProductID)
);
```

💡 “Composite Key ensures **no duplicate combinations**, not just duplicate values..

FOREIGN KEY Constraint

→ Creates a relationship between two tables- enforces referential integrity.

What is Referential Integrity?
Referential Integrity is a rule in SQL that ensures **relationships** between tables remain valid.
It makes sure that a FOREIGN KEY value in a child table always matches a PRIMARY KEY value in a parent table.

```
Syntax:
CREATE TABLE table_name (
    column_name datatype,
    FOREIGN KEY (column_name)
        REFERENCES parent_table(parent_column)
);
```

Example of Foreign Key

Step 1: Customers Table (Parent Table)

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
     City VARCHAR(50)
);
```

- **CustomerID is the PRIMARY KEY**
- **It uniquely identifies each customer.**

Step 2:Create Orders Table (Child Table)

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    OrderDate DATE,
    CustomerID INT,
    FOREIGN KEY (CustomerID)
    REFERENCES Customers(CustomerID)
);
```

- **CustomerID in Orders is a FOREIGN KEY**
- **It refers to CustomerID in Customers table.**

CHECK Constraint

Ensures values in a column meet a specific condition before they are accepted.

```
Syntax:
CREATE TABLE table_name(
  column_name datatype CHECK(Condition)
);
```

```
Example:
CREATE TABLE Employees(
      EmpId INT;
      Age INT CHECK(Age>=18)
);
```

💡 **Age must be 18 or more — values below 18 will be rejected by the database.**

DEFAULT Constraint

Automatically sets a default value for a column when no value is provided.

```
Syntax:
CREATE TABLE table_name (
column_name datatype DEFAULT default_value
);
```

```
EXAMPLE:
CREATE TABLE Employees (
    EmpID  INT,
    City   VARCHAR(50) DEFAULT 'Mumbai'
);
```

💡 **If City is not specified during INSERT, it automatically becomes 'Mumbai'.**

⭐ Master Example — All Constraints Together

```sql
CREATE TABLE Employees (
  EmpID    INT             PRIMARY KEY AUTO_INCREMENT,
  Name     VARCHAR(100)    NOT NULL,
  Email    VARCHAR(150)    NOT NULL UNIQUE,
  Age      INT             CHECK (Age >= 18 AND Age <= 65),
  Gender   CHAR(1)         CHECK (Gender IN ('M','F','O')),
  DeptID   INT,
  Status   VARCHAR(10)     DEFAULT 'Active',
  JoinDate TIMESTAMP       DEFAULT NOW(),
  FOREIGN KEY (DeptID)
    REFERENCES Departments(DeptID)
);
```

Constraints used:

✓ PRIMARY KEY

✓ AUTO_INCREMENT

✓ NOT NULL

✓ UNIQUE

✓ CHECK

✓ DEFAULT

✓ FOREIGN KEY

This single table applies ALL 6 constraints — professional-grade, bulletproof database design.

Advantages of SQL Constraints

✓

Maintain Data Accuracy

Ensures only valid, accurate data is stored in the database

✓

Prevent Invalid Entries

Automatically rejects data that violates defined rules

✓

Improve Database Security

Protects the database from unauthorized or incorrect modifications

✓

Ensure Referential Integrity

Maintains valid relationships between related tables

Conclusion

SQL Constraints are essential for:

- **Data Integrity**
- **Data Accuracy**
- **Reliable Database Systems**

Without constraints, databases become inconsistent and unreliable.

With constraints, your data remains clean, accurate, and trustworthy!

Presented by: Isha Landge